

Exploración de tipos de personalidades MBTI

1. Integrantes:

Agustina Igarreta, Emilia Welyzcko, Leila Mena y Delfina Guirao.

2. Objetivo y descripción general

El programa simula un test de personalidad inspirado en el modelo MBTI (Myers-Briggs Type Indicator). No es el test oficial del MBTI ni una herramienta de diagnóstico psicológico. Los resultados representan afinidades exploratorias calculadas sobre un dataset sintético.

Asimismo, le brinda al usuario el porcentaje de personas que comparten su mismo tipo así como también se compara su perfil con el promedio del grupo. De esta manera la persona puede situarse dentro de una distribución más amplia en lugar de recibir un resultado aislado.

El usuario debe responder un cuestionario a través de la consola. Con esta información el sistema calcula su afinidad porcentual hacia cada uno de los 8 polos de personalidad (E/I, S/N, T/F, J/P). Luego, determina su tipo a partir de los polos con mayor afinidad (por ejemplo, "ENTP"). Por último, compara ese perfil contra un dataset de más de 43.000 registros sintéticos y genera resultados con sus respectivas visualizaciones gráficas.

Flujo de ejecución (`programa\_principal.py`)

1. Se muestra un banner de bienvenida y un ****disclaimer ético**** (el MBTI no es un instrumento clínico validado).
2. Se carga el dataset (`datos/data.csv`). Si el archivo no existe o falla la carga, el programa avisa y continúa sin datos comparativos (no se rompe).
3. Se piden datos demográficos: nombre, edad (validada entre 14 y 99) y género (validado, debe estar dentro de tres opciones que se brindan).
4. Se presentan 20 preguntas (5 por cada una de las 4 dimensiones) con escala Likert de 1 a 5. Se valida que cada respuesta sea un número entre 1 y 5.
5. Se calculan los puntajes netos por dimensión, las afinidades porcentuales y el tipo MBTI que predomina.
6. Se calculan las siguientes métricas: distribución de tipos, intereses predominantes del tipo del usuario, promedios por tipo, ranking del tipo dentro del dataset y comparación usuario vs. grupo.
7. Se muestran los resultados finales en consola.
8. Si el dataset se cargó correctamente, se despliega un "menú interactivo" de visualizaciones (afinidades, género por tipo, intereses) que el usuario puede repetir tantas veces como quiera hasta decidir salir. Las opciones inválidas se manejan con un mensaje de error y vuelven a pedir la opción, sin romper el programa. Si el programa continuó con None(no se cargo ningún dataset) finaliza

Principales funcionalidades:

- Test interactivo de 20 preguntas con validación de cada respuesta.
- Cálculo de afinidades porcentuales y tipo MBTI de 4 letras.
- Comparación estadística del perfil del usuario contra un dataset real cargado con pandas.
- Generación de 3 tipos de gráficos distintos guardados automáticamente como imágenes.

- Manejo de errores (dataset faltante, entradas inválidas, opciones de menú fuera de rango) sin interrumpir la ejecución.

Distribución del trabajo:

El trabajo fue pensado, analizado y revisado en conjunto en su totalidad, pero ciertas tareas fueron profundizadas de manera exhaustiva por determinados integrantes para garantizar eficiencia. En primer lugar, cada integrante indago en busca de ideas y Datasets para proponer posibles trabajos y acordar en la elección de un mismo tema. Luego pensamos un prompt para IA. Ya teniendo el código, cada integrante se encargó de profundizar y revisar dos funciones específicas con el objetivo de garantizar que sea un código comprensible y acorde a nuestro nivel académico.

Tareas:

Delfina Guirao: analisis_dataset, resultados

Agustina Igarreta: metricas y preguntas

Leila Mena: carga de datos y validaciones

Emilia Welyczko: utilidades, diagrama de flujo del programa principal

Conjunto: gráficos, main, readme, presentación de canvas, prompt.

3. Descripción de la fuente de datos

- Archivo: `datos/data.csv`
- Cantidad de registros: 43.744 filas de datos
- Columnas: `Age`, `Gender`, `Education`, `Introversion Score`, `Sensing Score`, `Thinking Score`, `Judging Score`, `Interest`, `Personality`.
- Es un dataset sintético, no una muestra clínica real.
- El dataset fue descargado de la plataforma de kaggle
<https://www.kaggle.com/datasets/stealthtechnologies/predict-people-personality-types>

4. Instrucciones para ejecutar el programa

1. Clonar el repositorio

```
git clone https://github.com/DelfinaGuirao/Trabajo_Final_Aplicado.git
```

```
cd Trabajo_Final_Aplicado
```

2. Instalar las dependencias

```
pip install pandas matplotlib
```

3. Ejecutar el programa

```
python programa_principal.py
```

Luego seguir las instrucciones que aparecen en la consola (ingresar datos, responder el test y elegir gráficos del menú final).

Los gráficos generados se guardan automáticamente en `outputs/graficos/` y también se muestran en pantalla al momento de elegirlos en el menú.

5. Librerías

- **Pandas:** lectura del CSV, limpieza de datos y todos los cálculos estadísticos sobre el dataset (distribución de tipos, promedios, comparaciones).
- **Matplotlib.pyplot:** generación de los tres gráficos del sistema (torta de afinidades, barras de género, torta de intereses).

6. Estructura del repositorio

```
Trabajo_Final_Aplicado/
├── programa_principal.py    # Punto de inicio del programa
├── README.md
├── datos/
│   ├── data.csv            # Dataset de personalidades
├── diagramas/
│   ├── diagrama- mostrar_resultados_finales.docx
│   └── diagramas- analisis_dataset.docx
├── outputs/
│   └── graficos/           # Carpeta donde se guardan los .png generados al ejecutar el
programa
└── src/
    ├── carga_datos.py      # Lectura y limpieza del dataset
    ├── preguntas.py        # Set de preguntas del test
    ├── validaciones.py     # Validación de inputs del usuario
    ├── metricas.py         # Cálculo de puntajes, afinidades y tipo MBTI
    ├── analisis_dataset.py # Análisis comparativo con el dataset
    ├── resultados.py       # Presentación de resultados finales en consola
    ├── utilidades.py       # Funciones auxiliares de presentación (banner, separadores)
    └── graficos.py         # Generación de los gráficos
```

7. Clases implementadas

El proyecto no implementa clases. Todo el sistema está construido con funciones las cuales están organizadas en distintos módulos dentro de `src/` (programación funcional/procedural), según su responsabilidad,

8. Explicación breve de las funciones principales

- `src/carga_datos.py:`
- `cargar_dataset(ruta):` lee el CSV con pandas y lanza un error claro si el archivo no existe o si el formato no es válido.
- `limpiar_datos(df):` elimina filas con valores nulos en las columnas clave, normaliza el texto de `Personality` y convierte las columnas de puntajes a tipo numérico. Además ,descarta las filas que no se pueden convertir.

- src/validaciones.py:
 - validar_edad(), validar_genero(), validar_respuesta(): piden un dato por consola y repiten la solicitud hasta recibir una entrada válida. Hacen uso de `try/except` para que un valor no numérico no rompa el programa.
 - validar_rango(valor, minimo, maximo, nombre): valida un rango numérico, dato que se vuelve a utilizar en otras partes del código.
- src/metricas.py
 - calcular_scores(preguntas, respuestas): convierte las 20 respuestas (escala 1 a 5) en puntajes netos por dimensión.
 - calcular_afinidades(puntajes): transforma esos puntajes netos en porcentajes de 0% a 100% por cada polo.
 - determinar_tipo_mbti(puntajes): arma el tipo de 4 letras eligiendo, en cada dimensión, el polo con mayor afinidad.
 - obtener_top_tipos(puntajes, afinidades, top_n=5): calcula, mediante el promedio geométrico de las afinidades, los tipos MBTI más parecidos al perfil del usuario.
- src/analisis_dataset.py:
 - intereses_predominantes(df, tipo): intereses más frecuentes entre las personas del mismo tipo que el usuario.
 - calcular_rareza(df, tipo): cantidad, porcentaje y ranking de frecuencia del tipo del usuario dentro del dataset.
 - comparar_usuario_vs_grupo(df, tipo, scores_usuario): compara los puntajes del usuario contra el promedio de su grupo.
 - calcular_genero_por_tipo(df, tipo): porcentaje de cada género entre las personas del tipo del usuario.
- programa_principal.py
 - main(): función que llama, en orden, a todos los módulos anteriores y maneja el menú interactivo final de visualizaciones.

9. Resultados, salidas y gráficos generados

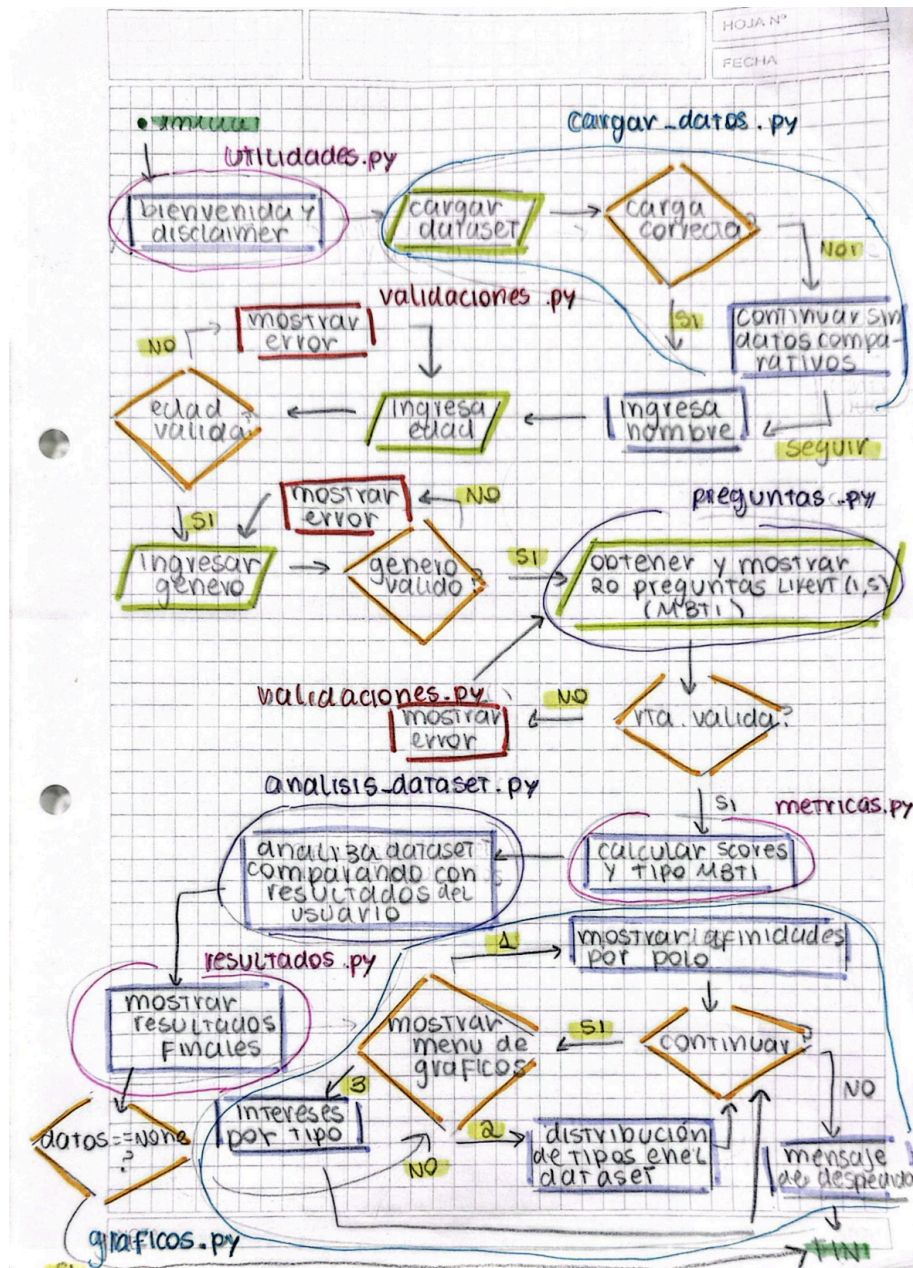
El programa muestra tres tipos de resultados:

1. Salida impresa en consola: tipo MBTI predominante con su descripción, afinidad porcentual de cada dimensión, top 5 de tipos más afines, ranking de frecuencia del tipo dentro del dataset ("rareza") e intereses predominantes del grupo.
2. Gráfico de torta de afinidades por letra (`outputs/graficos/graficar\_torta\_usuario.png`).
3. Gráfico de barras con la distribución de género dentro del tipo del usuario (`outputs/graficos/graficar\_genero\_por\_tipo.png`).
4. Gráfico de torta de intereses predominantes del tipo del usuario (`outputs/graficos/grafico\_intereses.png`).

10. Diagramas de diseño

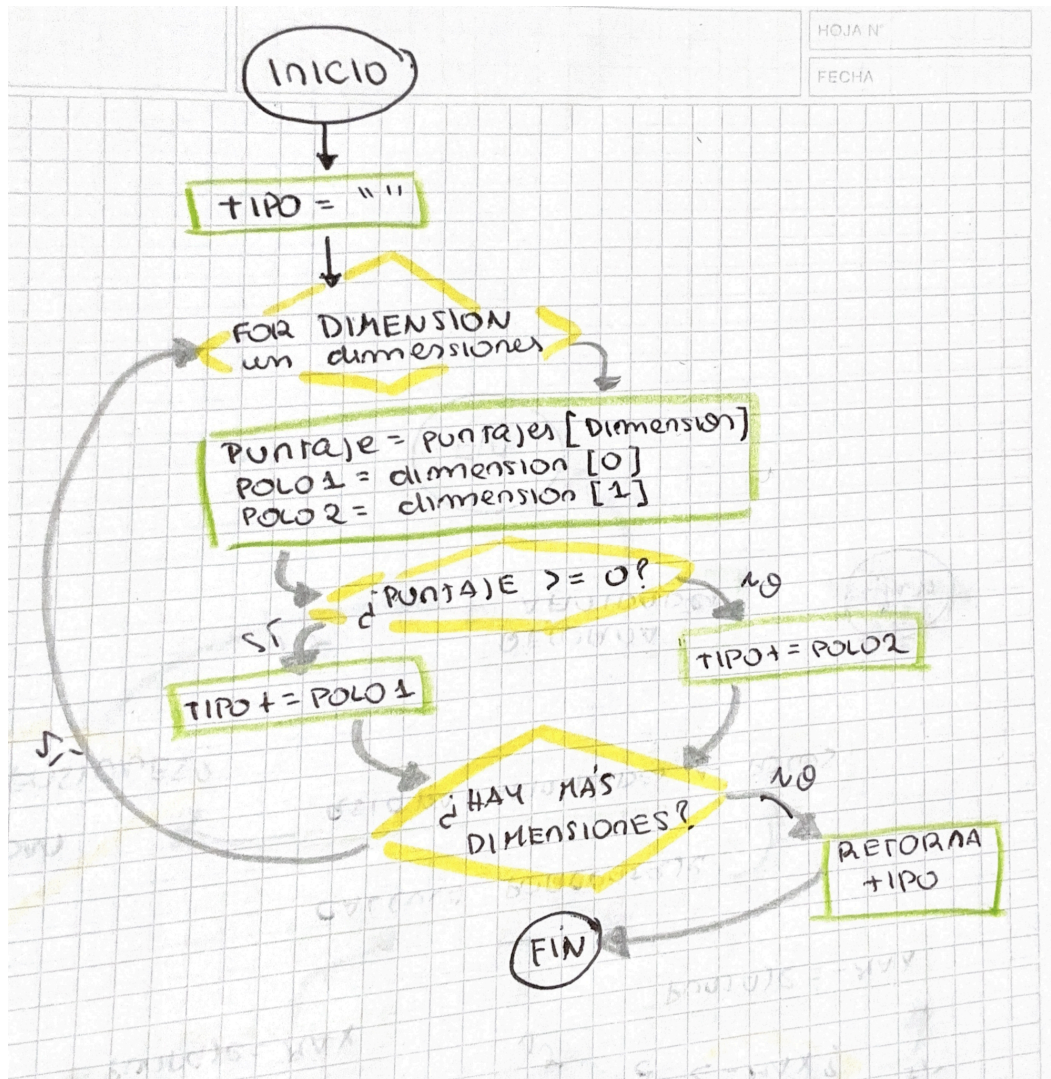
A continuación se presentan los diagramas de flujo que se consideraron importantes para comprender el flujo del programa. En la carpeta de diagramas se encuentran algunos que no son relevantes para entender el programa pero no está demás la aclaración de los mismos.

Programa principal

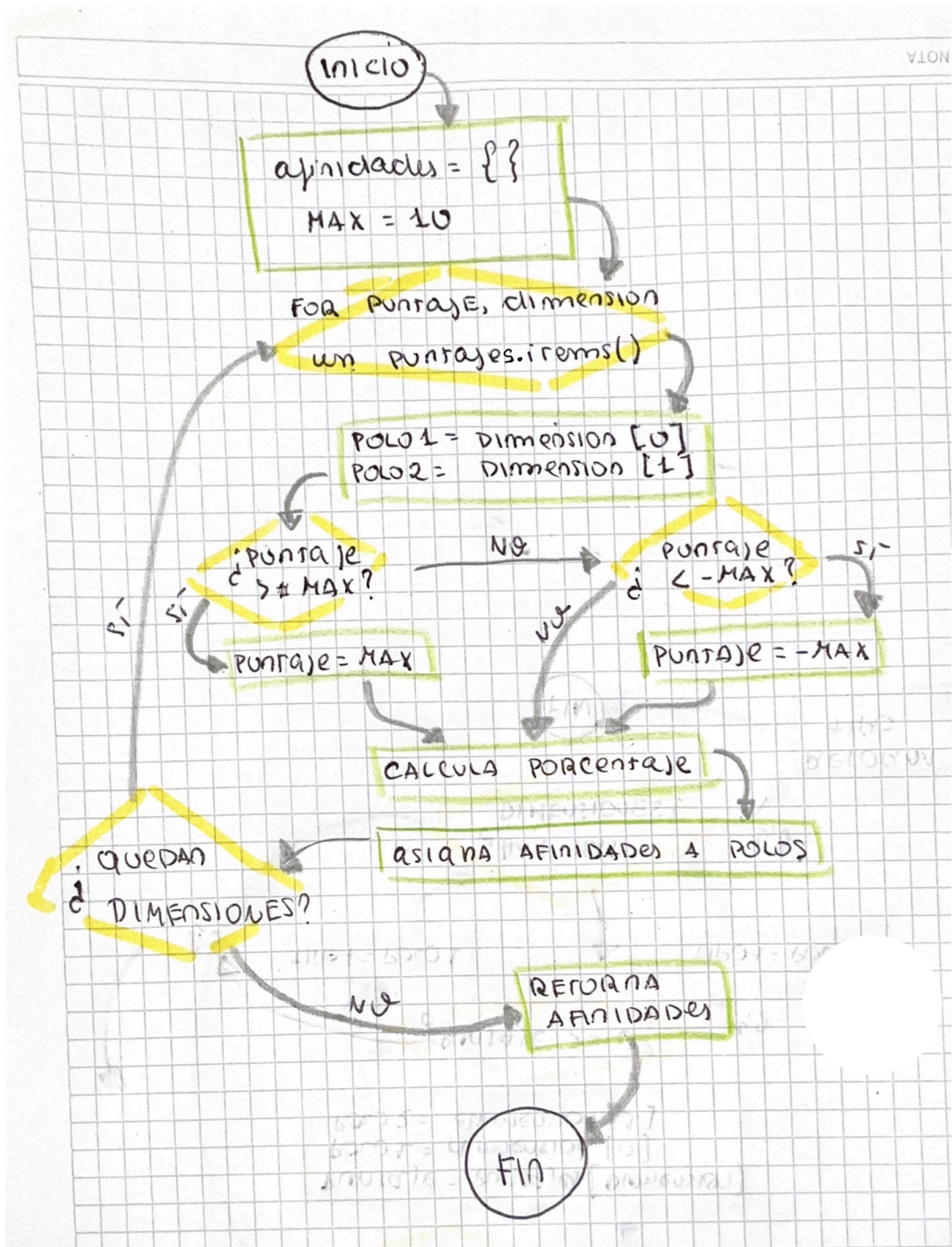


Métricas

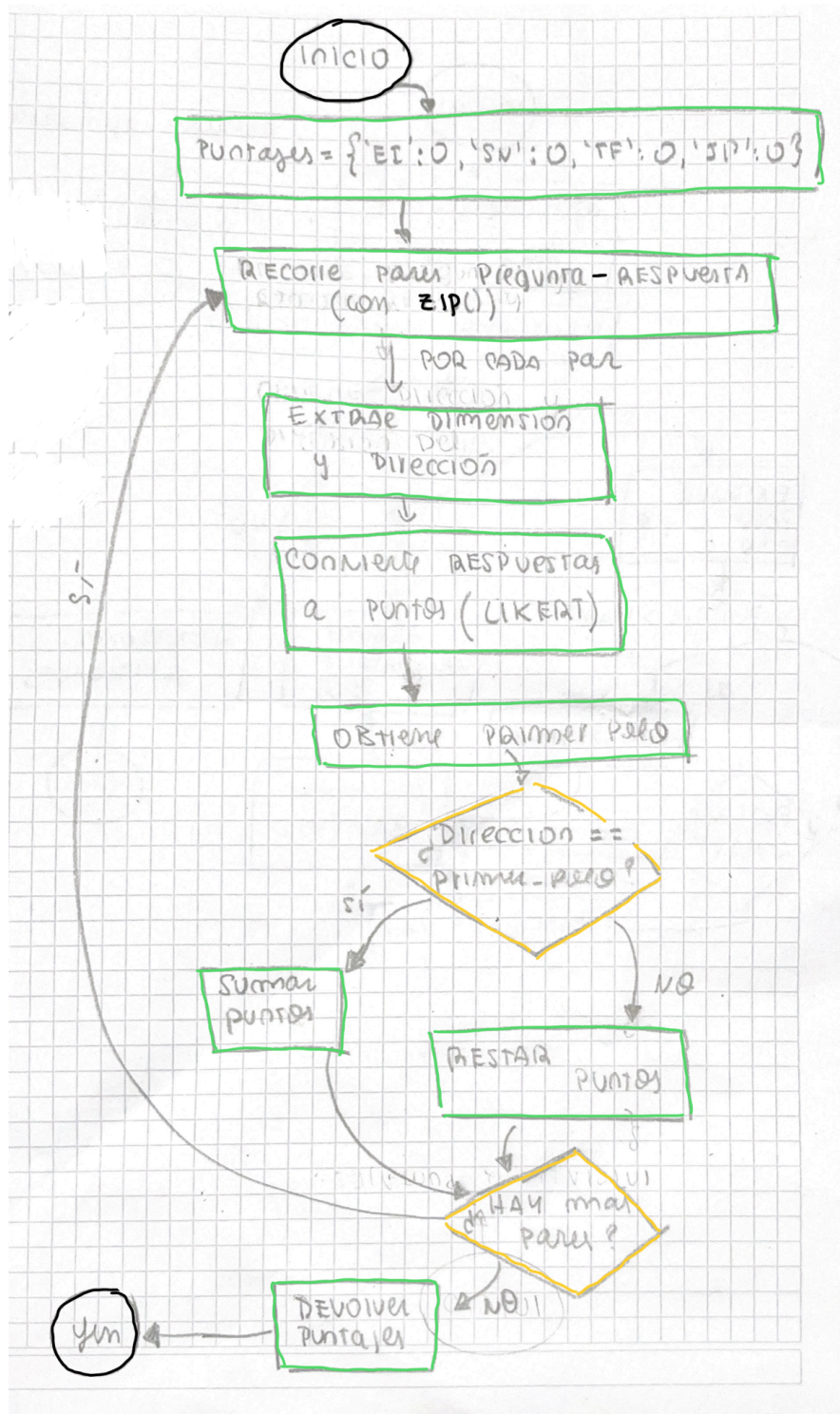
- determinar_tipo_mbti(puntajes)



- `calcular_afinidades(puntajes):`



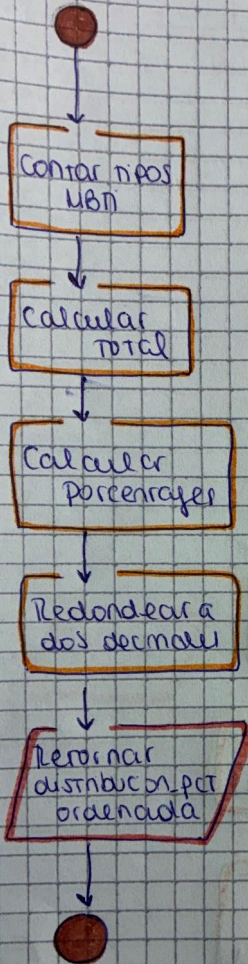
- `calcular_scores(preguntas,respuestas):`



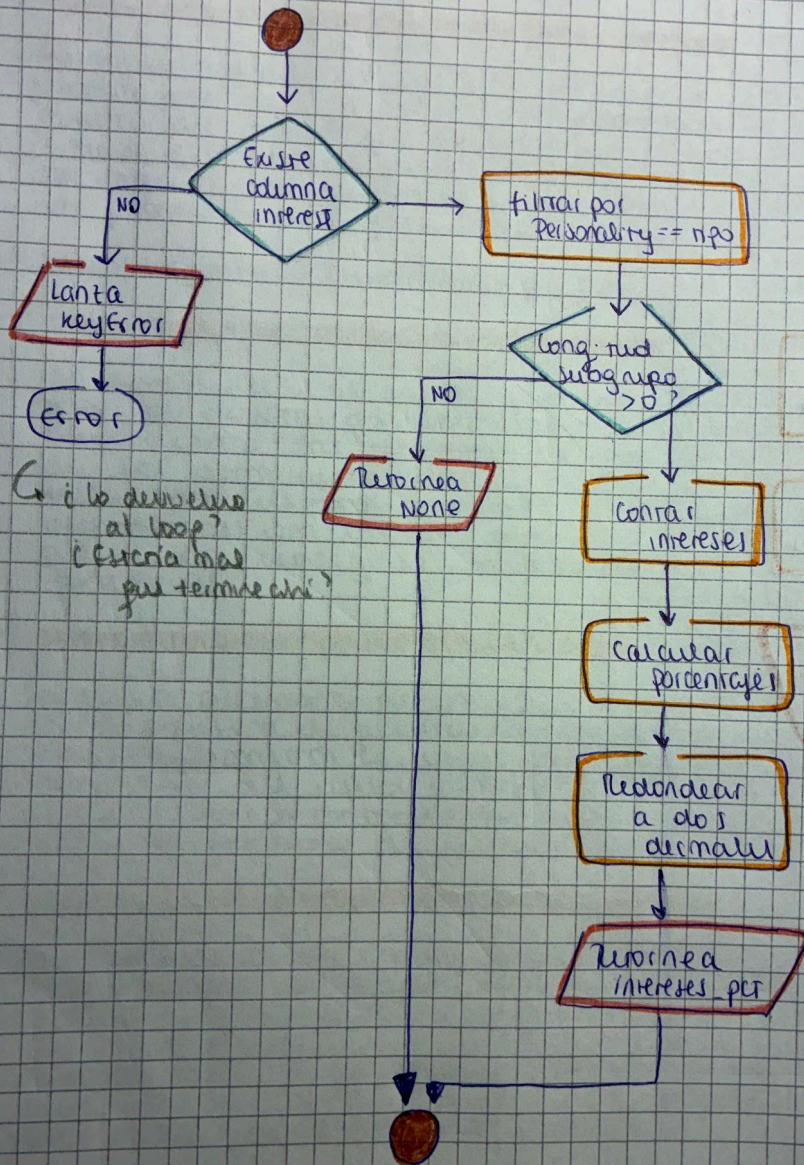
- intereses_predominantes_por_tipo
- calcular_rareza
- comparar_usuario_vs_grupo

Análisis dataset

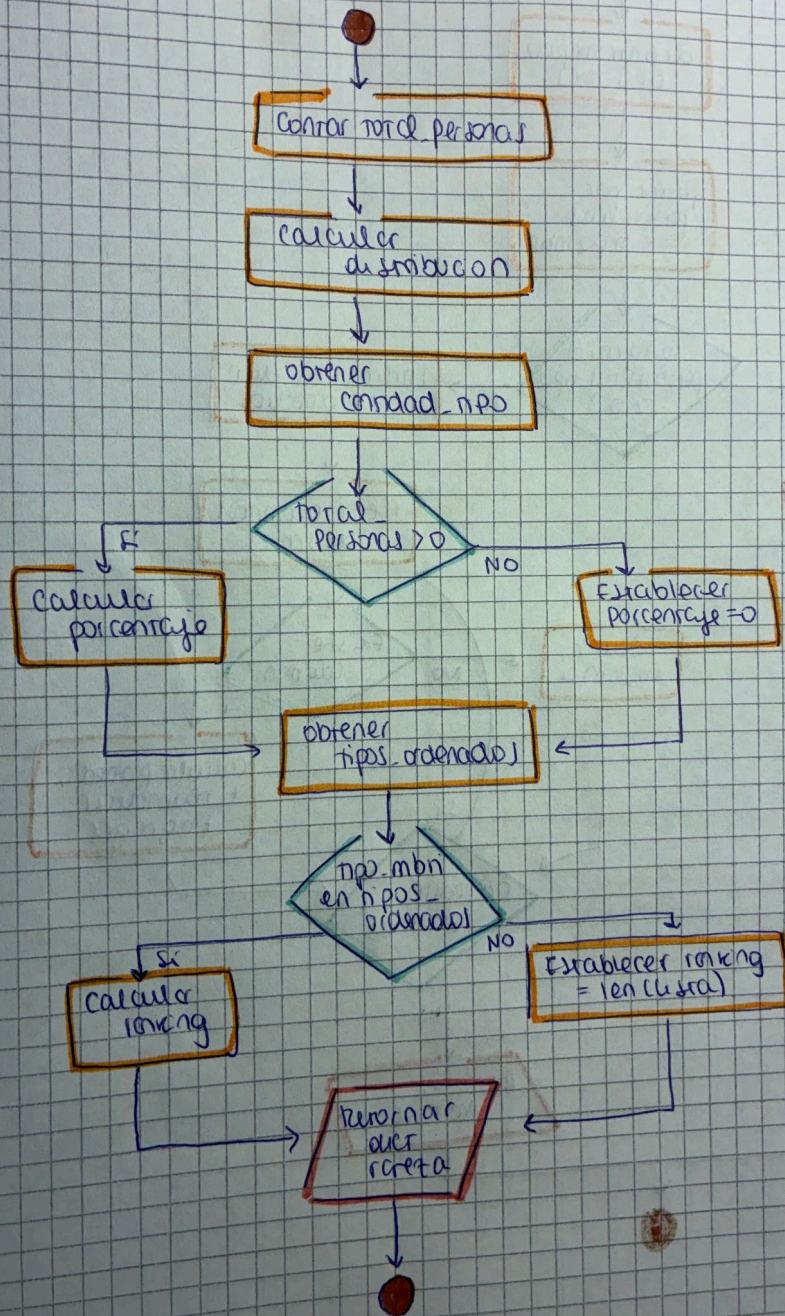
def calcular_distribucion_mbn(df):



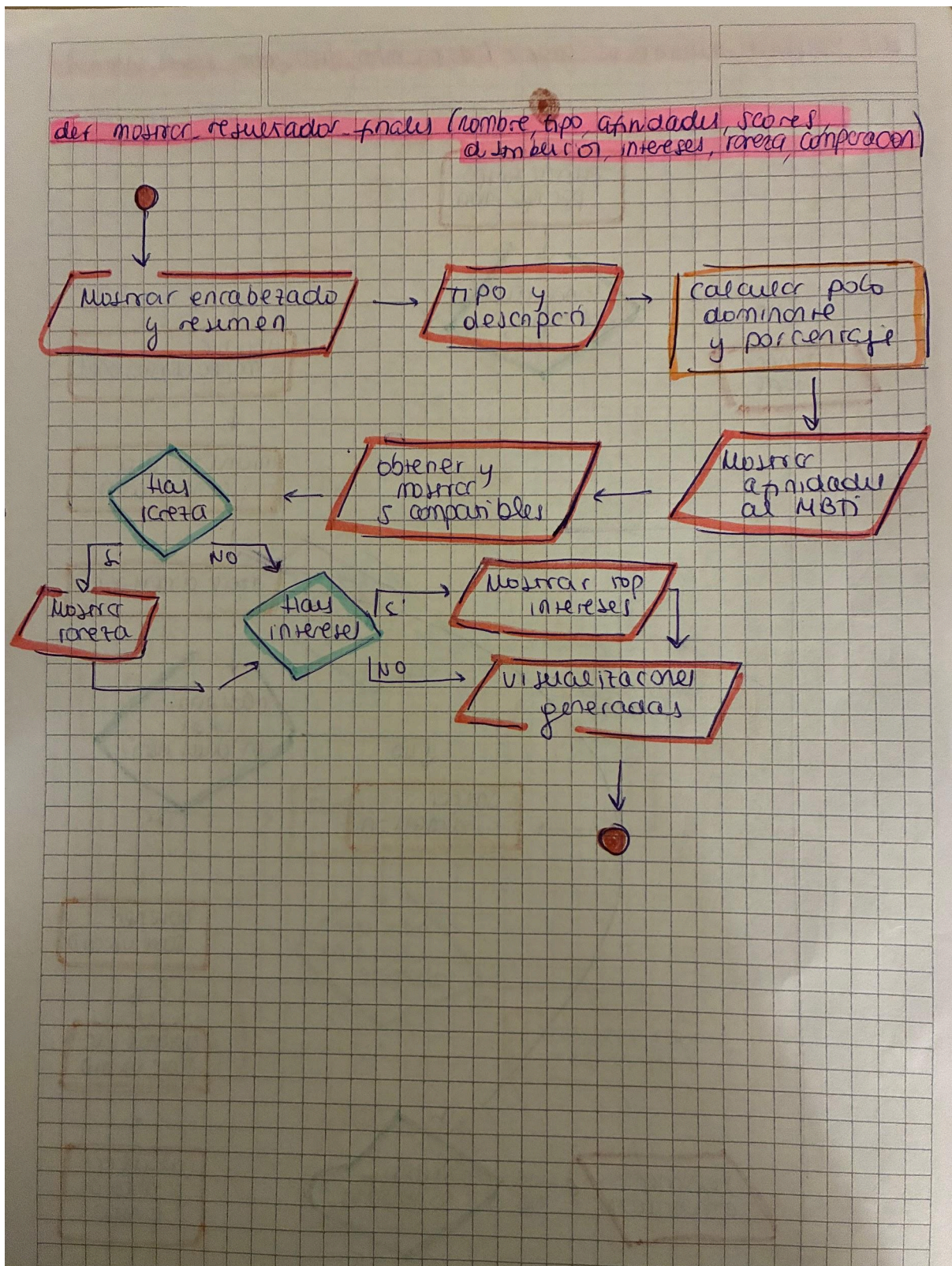
def intereses_pendientes(df, tipo):



def calcular_rareta (datos_mbn, n_pos_mbn):



def mostrar_resultados



11. Declaración de uso de IA

Hicimos uso de Chat Gpt en un principio para que nos ayude a organizar y distribuir todo el trabajo necesario que conlleva la creación del TA (trabajo aplicado). Asimismo, utilizamos esta misma IA para la división de módulos, creación de funciones, manejo de errores y visualizaciones. Todo este código que recibimos lo utilizamos como “base” y luego lo modificamos de manera que sea comprensible y acorde a nuestros conocimientos.

Prompt:

“Quiero que actúes como:

- experto en ciencias del comportamiento,
- arquitecto de software Python,
- docente universitario de programación,
- y mentor para estudiantes principiantes/intermedios en Python y pandas.

Necesito que me ayudes a desarrollar un proyecto final universitario COMPLETO, MODULAR, EXPLICABLE y DEFENDIBLE académicamente.

IMPORTANTE:

El proyecto NO debe presentarse como una herramienta psicológica clínica ni diagnóstica.

El enfoque debe ser: exploratorio, lúdico, interactivo, basado en afinidades, y estadístico/descriptivo.

NO usar lenguaje determinista como: "tu personalidad es...".

SÍ usar: afinidad, tendencias, similitud, compatibilidad, comparación poblacional, probabilidades.

CONTEXTO ACADÉMICO DEL TP

El trabajo final exige: Python modular, interacción con usuario, validaciones, manejo de errores, uso de pandas, mínimo 3 salidas o visualizaciones, documentación clara, estructura de carpetas, README, GitHub, división modular del código, y explicación defendible del sistema.

El usuario del proyecto es: personas mayores de 14 años interesadas en explorar afinidades MBTI.

DATASET UTILIZADO

Dataset sintético de 43.744 registros relacionado con MBTI (datos/data.csv).

Variables: Age, Gender, Education, Introversion Score, Sensing Score, Thinking Score, Judging Score, Interest, Personality.

Fuente: Kaggle (predict-people-personality-types, stealthtechnologies).

FEEDBACK DOCENTE A IMPLEMENTAR OBLIGATORIAMENTE:

1. Explicar cómo las respuestas se convierten en puntajes MBTI.
2. Mostrar afinidades/probabilidades y no verdades absolutas.
3. Definir claramente reglas de scoring.
4. Aprovechar el dataset: porcentaje de personas con ese tipo, intereses predominantes, comparación con promedio del grupo.
5. Agregar visualizaciones: distribución MBTI, intereses por tipo, usuario vs promedio, métricas descriptivas.
6. Validar respuestas.

7. Explicar qué representa la "población".
8. Aclarar limitaciones metodológicas y éticas.
9. Presentarlo como experiencia exploratoria/lúdica.

MODULARIZACIÓN SUGERIDA:

1. carga_datos.py — leer CSV, validar columnas, limpiar datos.
2. validaciones.py — validar entradas del usuario, manejo de errores, try/except.
3. analisis_dataset.py — análisis descriptivo del dataset.
4. visualizaciones.py — gráficos con matplotlib.
5. resultados.py — generar interpretación textual final.

REQUISITOS TÉCNICOS:

Lenguaje: Python.

Librerías: pandas, matplotlib, numpy (si es necesario).

Mantener: funciones simples, nombres claros, comentarios útiles, modularidad.

Agregar: docstrings, manejo de excepciones, validaciones robustas.

“

También usamos Miro IA para que nos genere diagramas de flujo y a partir de este contenido guiarnos y ayudarnos a hacer los diagramas propios.

A su vez, Claude nos ayudó a entender bien el funcionamiento del MBTI, a cómo evaluarlo, cómo interpretar las respuestas y calcular los porcentajes. Nos dio un código base para evaluar el test, que incluía las preguntas y cómo transformar las respuestas del usuario en el tipo de personalidad. Este código base lo fuimos modificando para nuestro entendimiento.

Por último, usamos Claude para que nos ayude a redactar el README del repositorio. Le compartimos el link al repositorio de GitHub, nuestro archivo de diseño, el prompt que habíamos usado con ChatGPT y la consigna completa con los puntos que debía incluir el README. A partir de esa información, Claude generó una primera versión de cada sección. Nosotras revisamos y corregimos el contenido que no se ajustaba a la realidad del proyecto (por ejemplo, partes del código que habían cambiado respecto al diseño original) antes de dejarlo como versión final.

Prompt:

”A partir de la información de este repositorio:

(https://github.com/DelfinaGuirao/Trabajo_Final_Aplicado/blob/main/diagramas/validar%20datos%20diagrama.pdf) y teniendo en cuenta los requisitos del Readme que plantea el pdf (insertamos el pdf con la consigna del TA), genera el contenido necesario para cada requisito de manera que nos guíe y ordene para generar nuestro propio Readme”.

12. Notas adicionales

La división del trabajo no se ve reflejada en todos los commits, ya que en ocasiones trabajamos con alguna computadora de otra integrante (por motivos de organización).